Artificial Intelligent

Neural Networks

Do Thanh Cong, congdt@vnu.edu.vn

Computer Science Department, Faculty of IT

VNU-UET

Table of content

- Perceptron
- MLP-Architecture
- MLP-Learning
- Deep MLP

Machine Learning – Ex.1



(1) x=3 → y=5

(2) x=10 → y=19

(3) x=-1 → y=-3

x=7 → y=?

Machine Learning – Ex.2



(1) $x_1 = 3,\ x_2 = 3 \rightarrow y = 6$

(2) $x_1 = 1,\ x_2 = 2 \rightarrow y = -1$

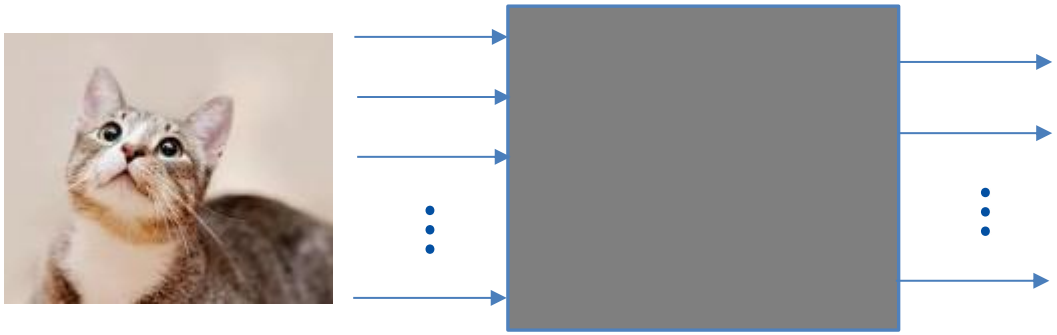
(3) $x_1 = 0,\ x_2 = 0 \rightarrow y = 0$

(4) $x_1 = 5,\ x_2 = -1 \rightarrow y = 26$

$x_1 = 100,\ x_2 = 200 \rightarrow y = ?$

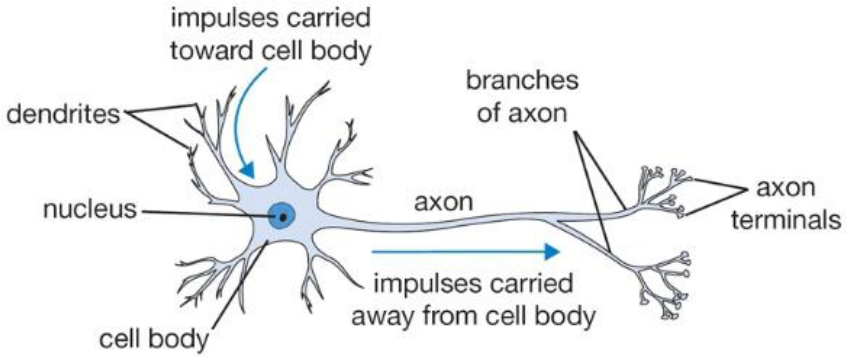
Machine Learning – Ex.3

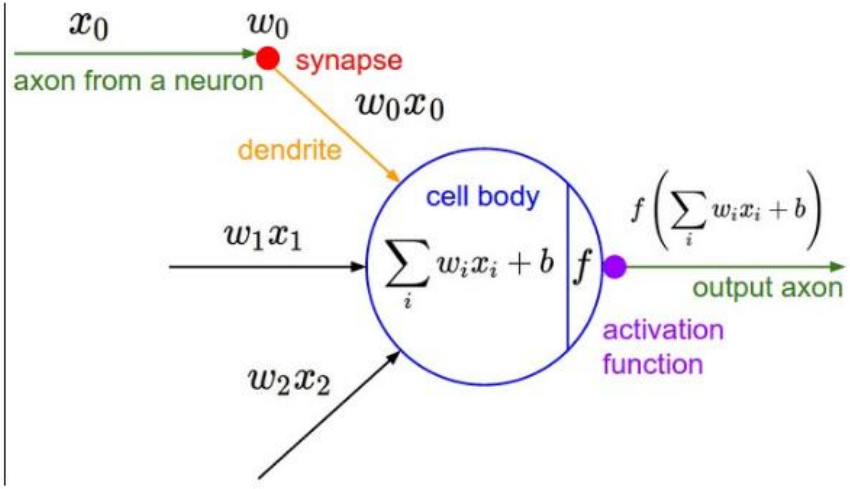




Perceptron – Architecture

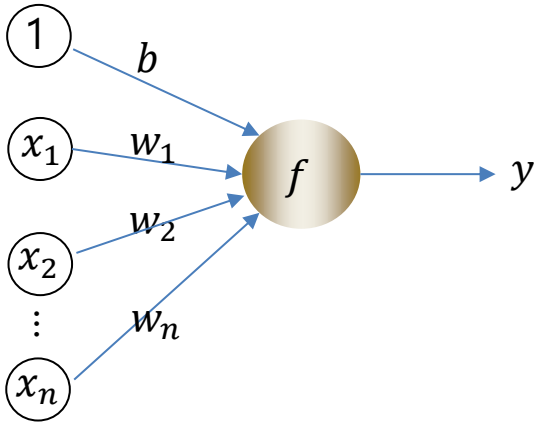
- **Biological neuron vs Artificial neuron**





Perceptron

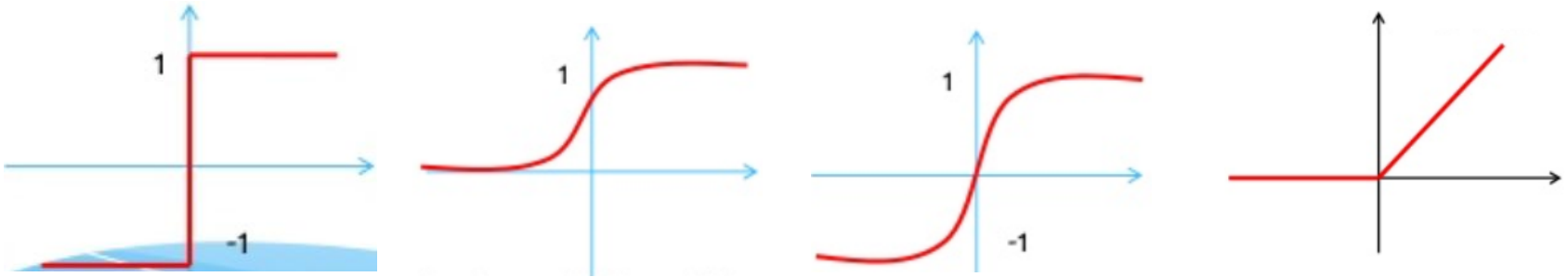
- **Architecture**



- input: $\mathbf{x}=[x_1, x_2, \ldots, x_n]^T$
- weights: $\mathbf{w}=[w_1, w_2, \ldots, w_n]^T$
- bias: b
- activation function: f
- output: $y = f(\mathbf{w}^T\mathbf{x} + b) = f(w_1x_1 + w_2x_2 + \cdots + w_nx_n + b)$

Activation Functions

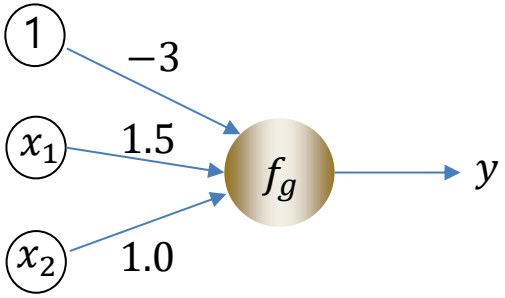
- **Activation functions for** $y = f(\mathbf{w}^T\mathbf{x} + b) = f(z)$
  - **sign:** $f_g(z) = \begin{cases} +1, & z \geq 0 \\ -1, & z < 0 \end{cases}$
  - **sigmoid:** $f_s(z) = \frac{1}{1+e^{-z}}, \ 0 \leq f \leq 1$
  - **tanh:** $f_t(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \ -1 \leq f \leq 1$
  - **ReLU:** $f_R(z) = max(0, z), \ 0 \leq f \leq \infty$

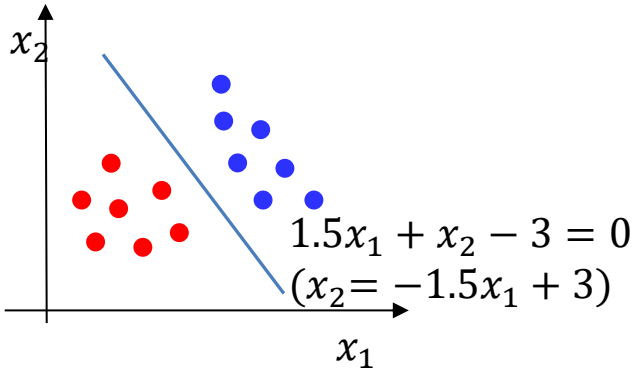


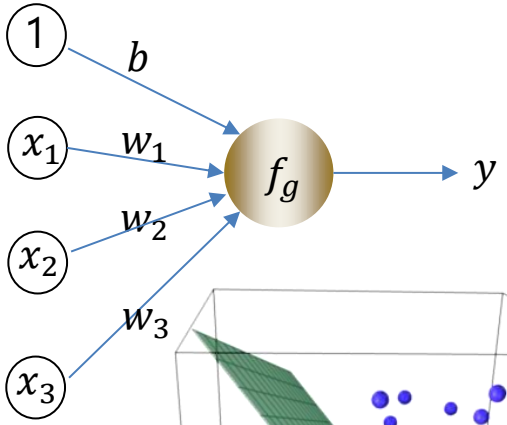
sign **sigmoid** **tanh** **ReLU**

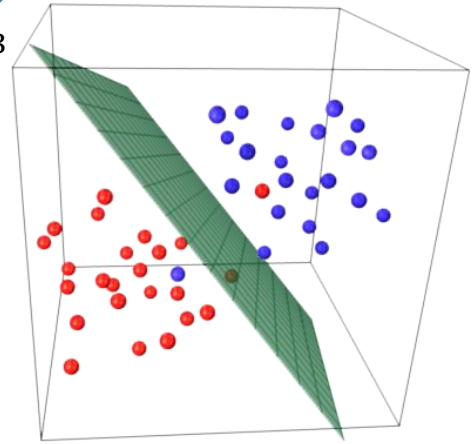
Perceptron

- **Single Perceptron with f_g represents a linear classifier in R^n**
  - $y = f_g(\mathbf{w}^T\mathbf{x} + b) = \begin{cases} +1, & \mathbf{w}^T\mathbf{x} + b \geq 0 \\ -1, & \mathbf{w}^T\mathbf{x} + b < 0 \end{cases}$





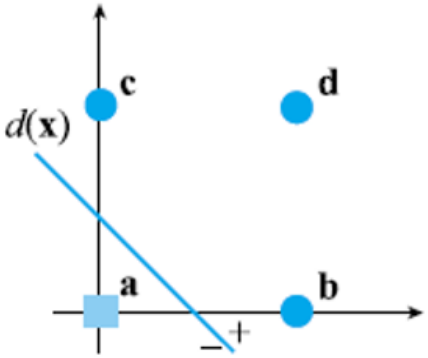


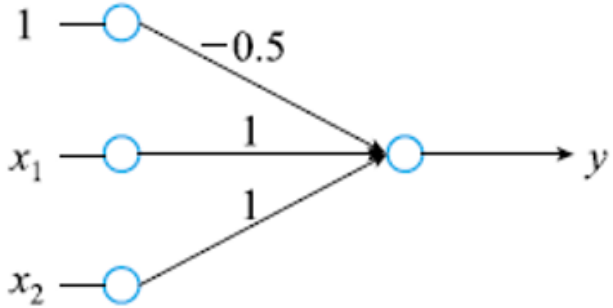


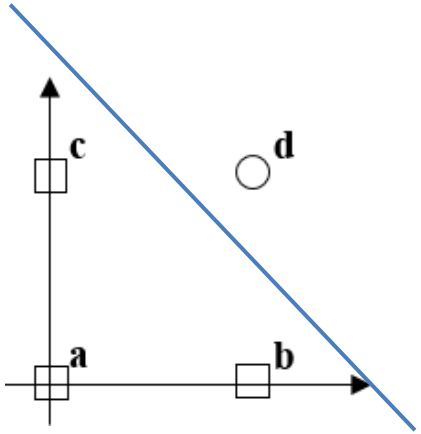
$w_1x_1 + w_2x_2 + w_3x_3 + b = 0$

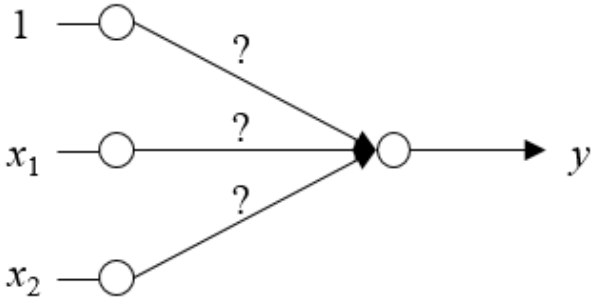
Perceptron - Capability

- **Logic functions: AND, OR, etc**









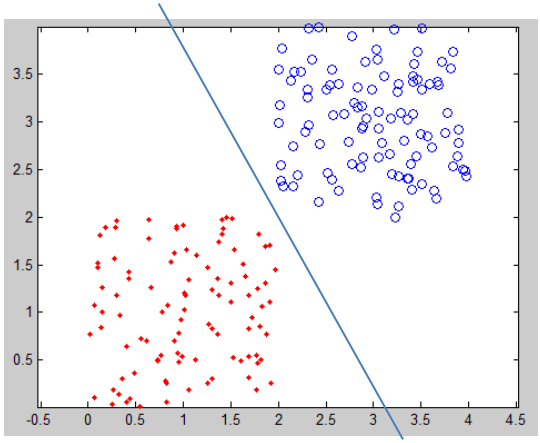
Perceptron – Learning

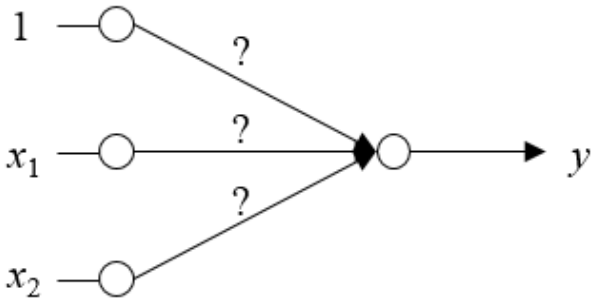
- **Problem definition: given a set of N training samples,**

  $\mathbf{X}=\{ (\mathbf{x}_1, t_1), (\mathbf{x}_2, t_2), \ldots, (\mathbf{x}_N, t_N) \}$,

  where $\mathbf{x}_i = [x_{i,1}, x_{i,2}, \ldots, x_{i,n}]^T$ and $t_i \in \{+1, -1\}$

☞ **find *a possible* $\mathbf{w}=[w_1, w_2, \ldots, w_n]^T$ and b which separates samples with $t = +1$ from samples with $t = -1$**



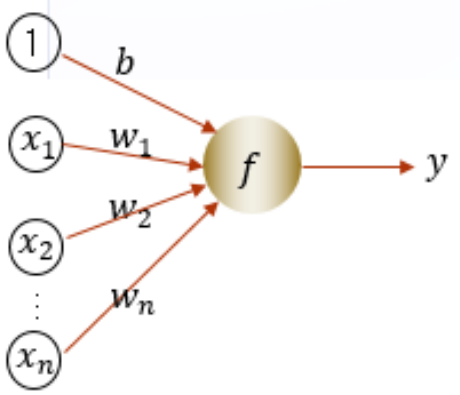


Perceptron – Learning

- **Given a $(\mathbf{x}_k, t_k)$, test the goodness for $\theta = \{\mathbf{w}, b\}$**

$t_k = +1$	$\mathrm{w}^{\mathrm{T}}\mathrm{x}_k + b \geq 0$	$y = +1$	correct
	$\mathrm{w}^{\mathrm{T}}\mathrm{x}_k + b < 0$	$y = -1$	error
$t_k = -1$	$\mathrm{w}^{\mathrm{T}}\mathrm{x}_k + b \geq 0$	$y = +1$	error
	$\mathrm{w}^{\mathrm{T}}\mathrm{x}_k + b < 0$	$y = -1$	correct

(*) assume $f = f_g$



- **Cost(Error) function for the goodness of $\theta = \{\mathbf{w}, b\}$**

$$J(\theta) = \sum_{\mathbf{x}_k \in Y} (-t_k)\,(\mathbf{w}^{\mathrm{T}}\mathbf{x}_k + b)$$

where Y is set of mis-classified samples

Recap: Perceptron

- **Gradient Descent Algorithm to find the optimal $\theta = \{\mathbf{w}, b\}$**

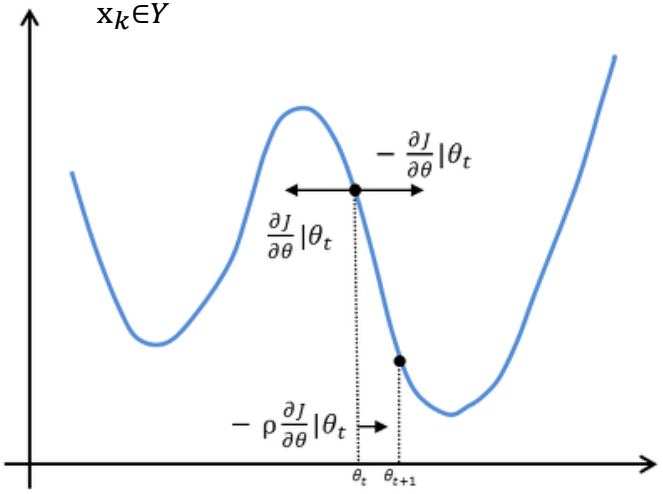
  - **initialize θ as $\theta(0)$ randomly**
  - **iterates**

$$\theta(t+1) = \theta(t) - \rho \frac{\partial J}{\partial \theta}$$

☞ $$\mathbf{w}(t+1) = \mathbf{w}(t) - \rho \sum_{\mathbf{x}_k \in Y} (-t_k)\,(\mathbf{x}_k)$$

☞ $$b(t+1) = b(t) - \rho \sum_{\mathbf{x}_k \in Y} (-t_k)$$

$$J(\theta) = \sum_{\mathrm{x}_k \in Y} (-t_k)(\mathbf{w}^T \mathbf{x}_k + b)$$

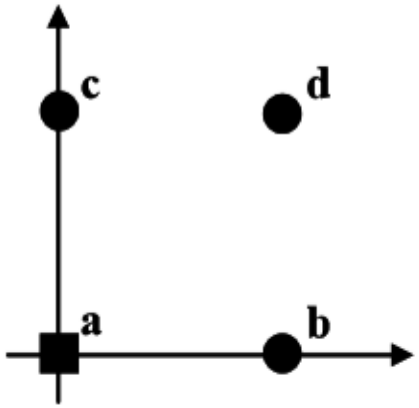


where ρ is a learning rate ($0 < \rho \ll 1$), and $\frac{\partial J}{\partial \theta}$ is the gradient

  - **until convergence to a local minimum**

Perceptron Learning Example (1)

Given the following information:



Training Samples

$\mathbf{a}=(0,0)^T$, $t_a=-1$

$\mathbf{b}=(1,0)^T$, $t_b=1$

$\mathbf{c}=(0,1)^T$, $t_c=1$

$\mathbf{d}=(1,1)^T$, $t_d=1$

Initialization

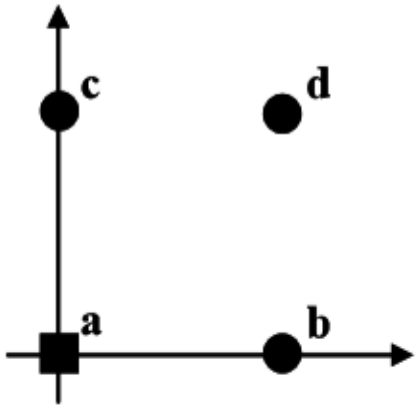
$\rho=0.4$

$\mathbf{w}(0)=(-0.5,0.75)^T$, $b(0)=0.375$ ①

Find **w** and **b** such that the perceptron correctly classifies all training samples?

Perceptron Learning Example (1)

Given the following information:



Training Samples

$\mathbf{a}=(0,0)^T$, $t_a=-1$

$\mathbf{b}=(1,0)^T$, $t_b=1$

$\mathbf{c}=(0,1)^T$, $t_c=1$

$\mathbf{d}=(1,1)^T$, $t_d=1$

Initialization

$\rho=0.4$

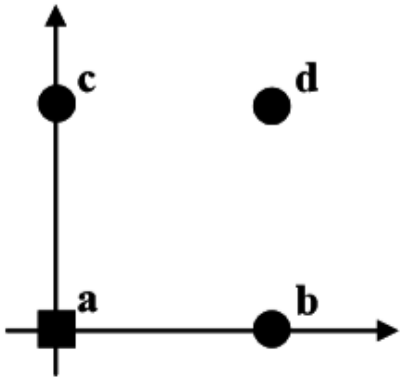
$\mathbf{w}(0)=(-0.5,0.75)^T$, $b(0)=0.375$ ①

Find **w** and **b** such that the perceptron correctly classifies all training samples?

☞ $$\mathbf{w}(t+1) = \mathbf{w}(t) - \rho \sum_{\mathbf{x}_k \in Y} (-t_k)\,(\mathbf{x}_k)$$

☞ $$b(t+1) = b(t) - \rho \sum_{\mathbf{x}_k \in Y} (-t_k)$$

Perceptron Learning Example



Training Samples

$\mathbf{a}=(0,0)^T$, $t_a=-1$

$\mathbf{b}=(1,0)^T$, $t_b=1$

$\mathbf{c}=(0,1)^T$, $t_c=1$

$\mathbf{d}=(1,1)^T$, $t_d=1$

Initialization

$\rho=0.4$

$\mathbf{w}(0)=(-0.5,0.75)^T$, $b(0)=0.375$ ①

Error Patterns

$d(\mathbf{x})=-0.5x_1+0.75x_2+0.375$

$Y=\{\mathbf{a}, \mathbf{b}\}$

$d(\mathbf{x})=-0.1x_1+0.75x_2+0.375$

$Y=\{\mathbf{a}\}$

Weight Updates

$$\mathbf{w}(1)=\mathbf{w}(0)+0.4(t_a\cdot\mathbf{a}+t_b\cdot\mathbf{b})=\begin{pmatrix}-0.5\\0.75\end{pmatrix}+0.4\left[-\begin{pmatrix}0\\0\end{pmatrix}+\begin{pmatrix}1\\0\end{pmatrix}\right]=\begin{pmatrix}-0.1\\0.75\end{pmatrix}$$ ②

$$b(1)=b(0)+0.4(t_a+t_b)=0.375+0.4*0=0.375$$

$$\mathbf{w}(2)=\mathbf{w}(1)+0.4(t_a\mathbf{a})=\begin{pmatrix}-0.1\\0.75\end{pmatrix}+0.4\left[-\begin{pmatrix}0\\0\end{pmatrix}\right]=\begin{pmatrix}-0.1\\0.75\end{pmatrix}$$ ③

$$b(2)=b(1)+0.4(t_a)=0.375-0.4=-0.025$$

Perceptron Learning Example

$d(\mathbf{x})$=-0.1x_1+0.75x_2-0.025

Y={**b**}

$$\mathbf{w}(3) = \mathbf{w}(2) + 0.4(t_b\mathbf{b}) = \begin{pmatrix} -0.1 \\ 0.75 \end{pmatrix} + 0.4\left[\begin{pmatrix} 1 \\ 0 \end{pmatrix}\right] = \begin{pmatrix} 0.3 \\ 0.75 \end{pmatrix}$$

$$b(3) = b(2) + 0.4(t_b) = -0.025 + 0.4 = 0.375$$

④

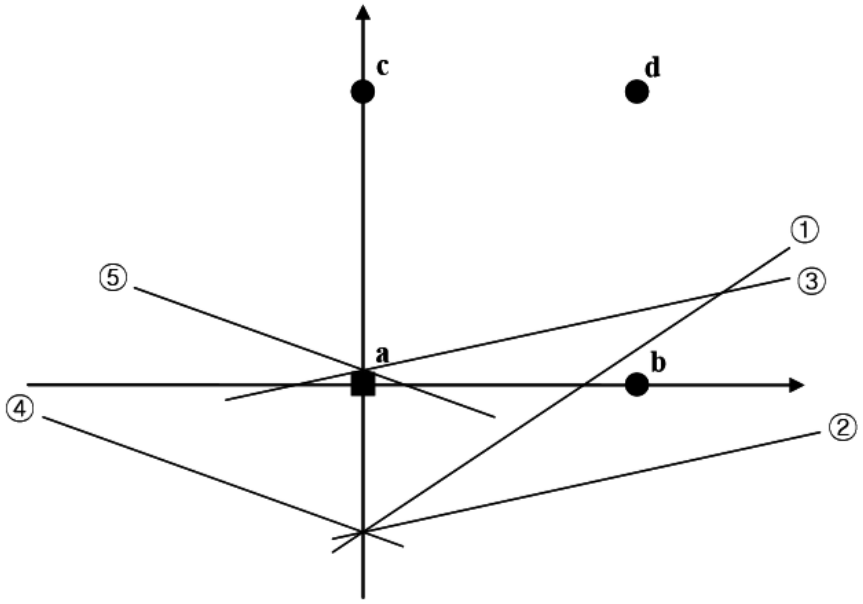
$d(\mathbf{x})$=-0.3x_1+0.75x_2+0.375

Y={**a**}

$$\mathbf{w}(4) = \mathbf{w}(3) + 0.4(t_a\mathbf{a}) = \begin{pmatrix} 0.3 \\ 0.75 \end{pmatrix} + 0.4\left[-\begin{pmatrix} 0 \\ 0 \end{pmatrix}\right] = \begin{pmatrix} 0.3 \\ 0.75 \end{pmatrix}$$

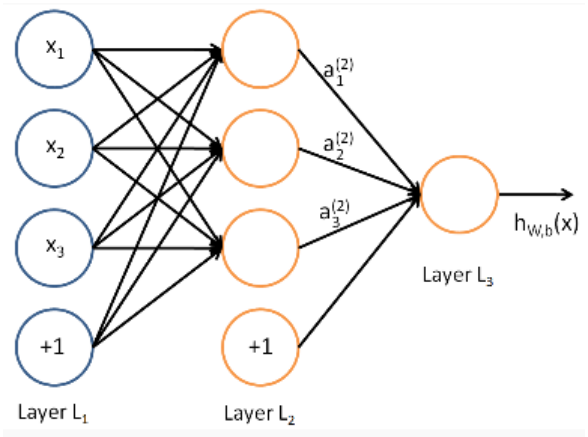
$$b(4) = b(3) + 0.4(t_a) = 0.375 - 0.4 = -0.025$$

⑤

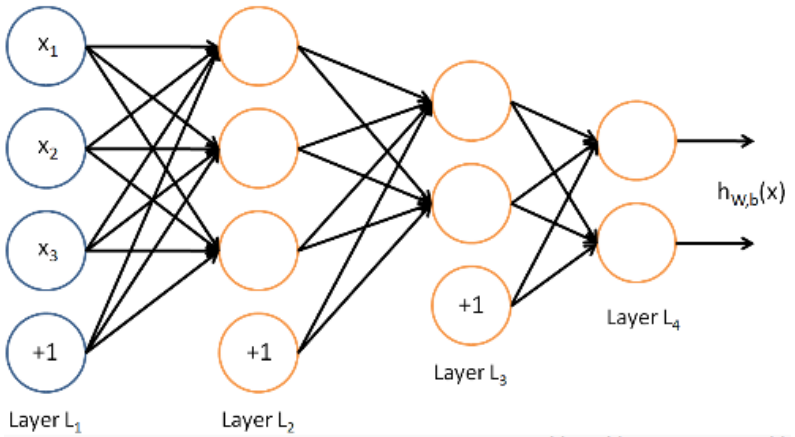


MLP – Architecture

- **Multi-Layer Perceptron**
  - input layer, hidden layer(s), output layer
  - feed-forward
  - fully-connected



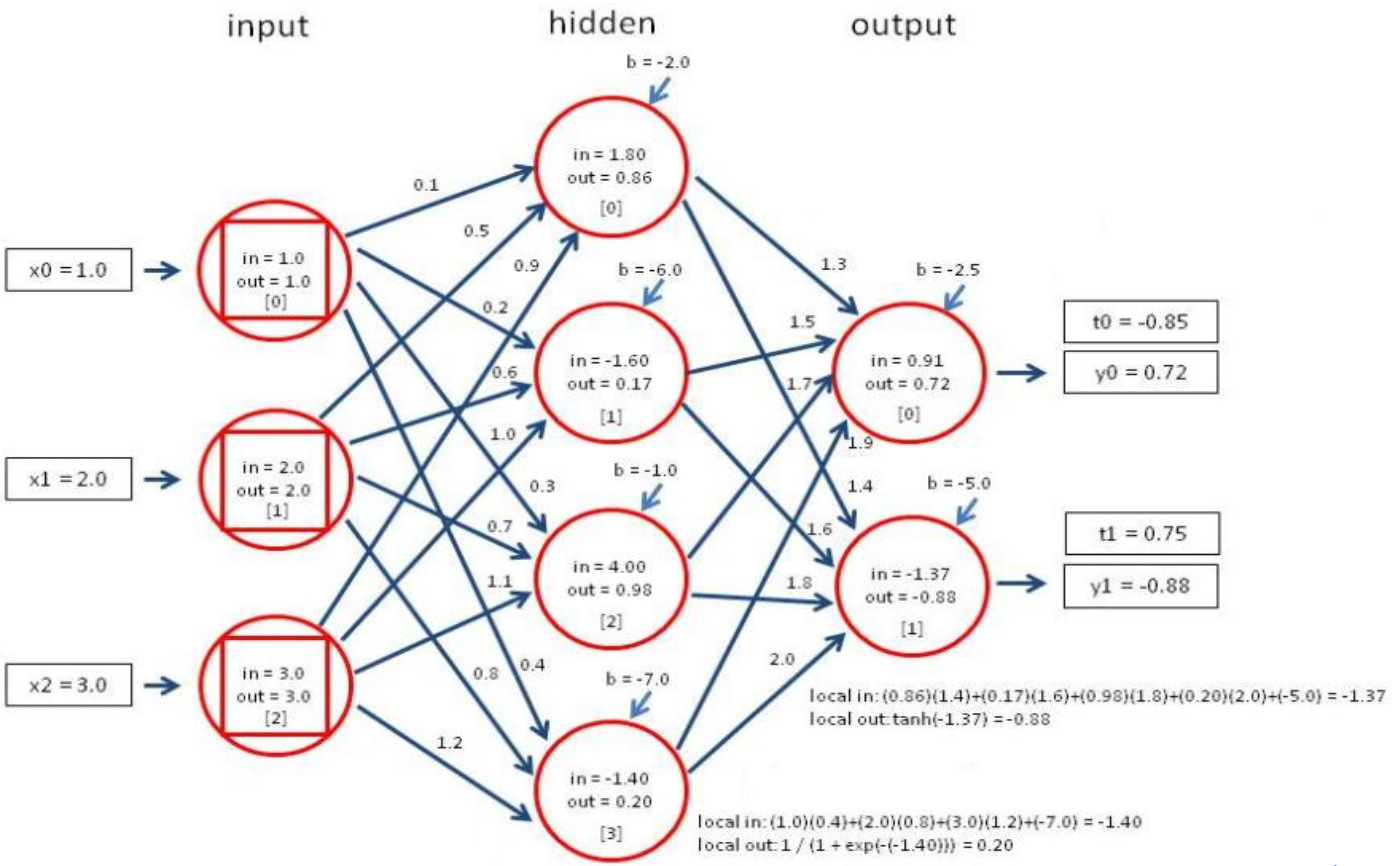
3×3×1 MLP



3×3×2×2 MLP

MLP – Architecture

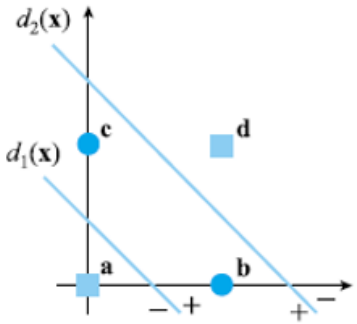
- **Forward calculations**

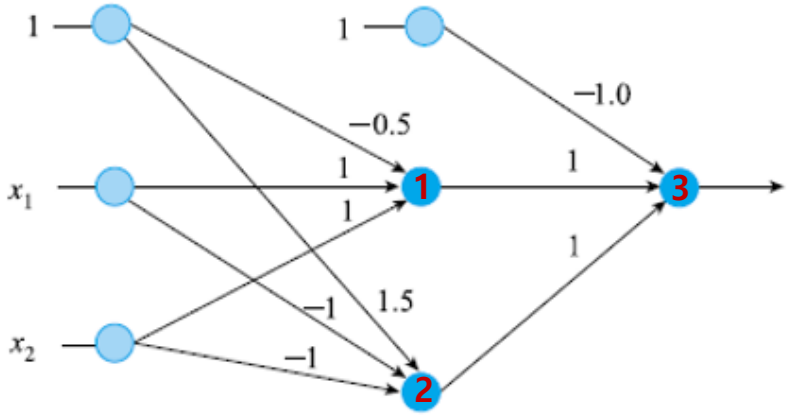


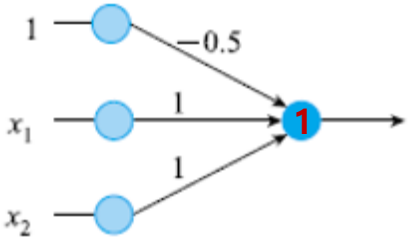
(*) assume $f = f_s(z) = \frac{1}{1+e^{-z}}$

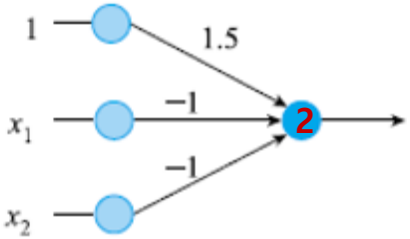
MLP – Capability

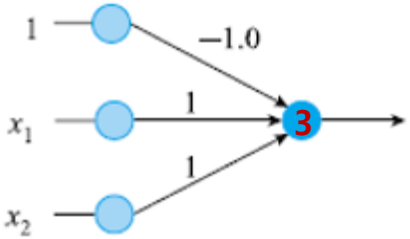
- **XOR function (sign activation, f_g)**





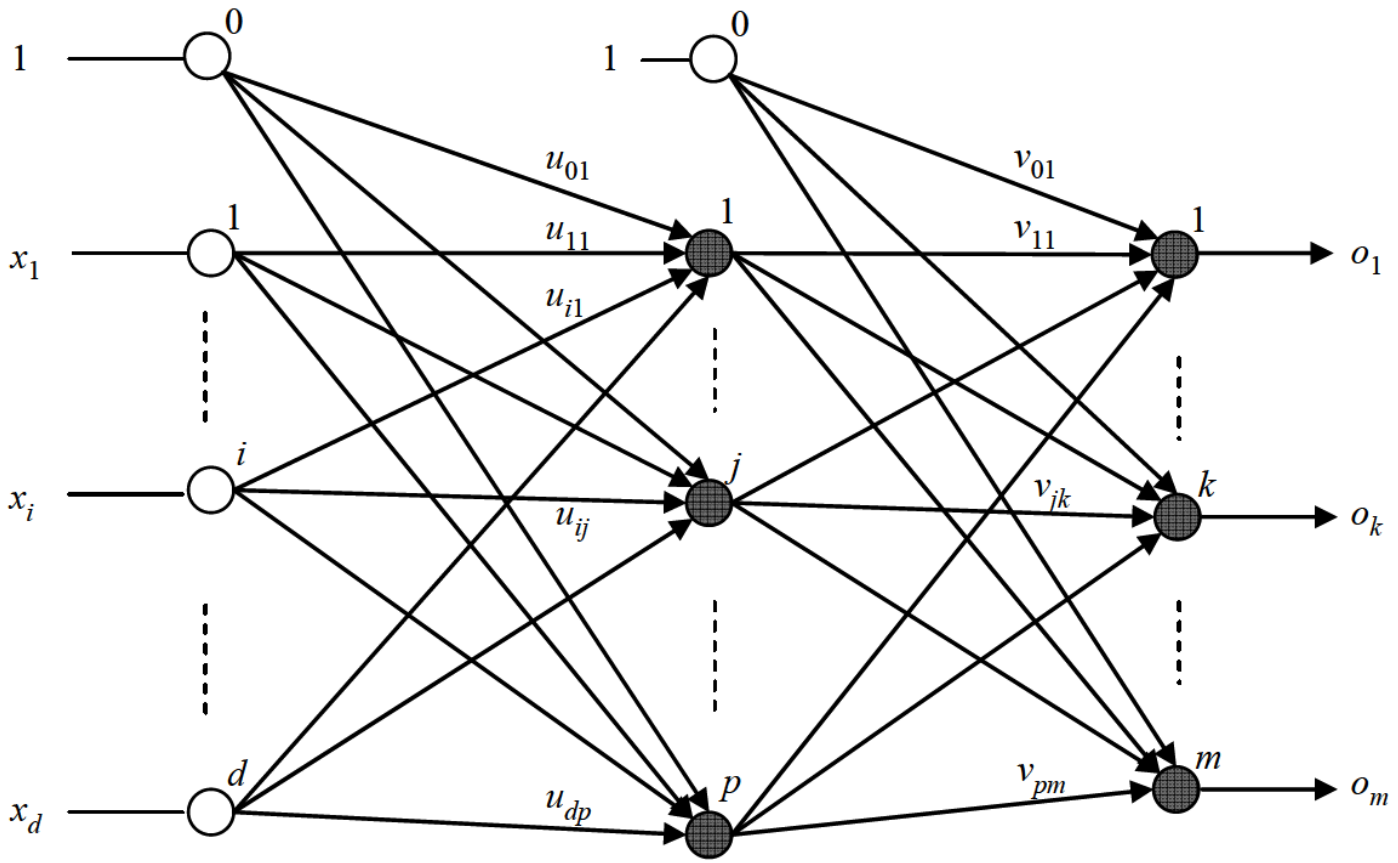






MLP Learning: 2-layer MLP

- **Weight parameters in a $d \times p \times m$ MLP**



MLP – Learning

- **Problem definition: given a set of N training samples,**

  $\mathbf{X}=\{ (\mathbf{x}_1, \mathbf{t}_1), (\mathbf{x}_2, \mathbf{t}_2), \ldots, (\mathbf{x}_N, \mathbf{t}_N) \}$,

  where $\mathbf{x}_i = [x_{i,1}, x_{i,2}, \ldots, x_{i,d}]^T$ and $\mathbf{t}_i = [t_{i,1}, t_{i,2}, \ldots, t_{i,m}]^T$

☞ **find *a possible* u and v which map $\mathbf{x}_i$ to the corresponding $\mathbf{t}_i$.**

- **Cost function for the goodness of $\theta = \{\mathbf{u}, \mathbf{v}\}$**

$$E(\theta) = \frac{1}{2}\sum_{k=1}^{m}(o_k - t_k)^2$$

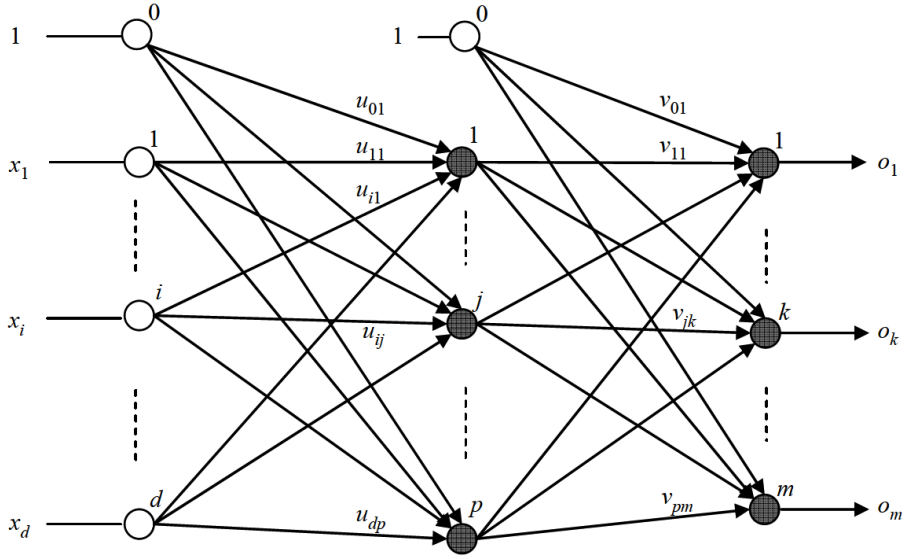
where m is the number of output nodes

MLP - Learning

- **Step 1: Forward calculations with a training sample x**
  - **input/output of j-th hidden node**

$$z_sum_j = \sum_{i=1}^{d} x_i\, u_{ij} + u_{0j} = \sum_{i=0}^{d} x_i\, u_{ij}$$

$$z_j = \tau_1(z_sum_j) = \tau_1\left(\sum_{i=0}^{d} x_i\, u_{ij}\right)$$



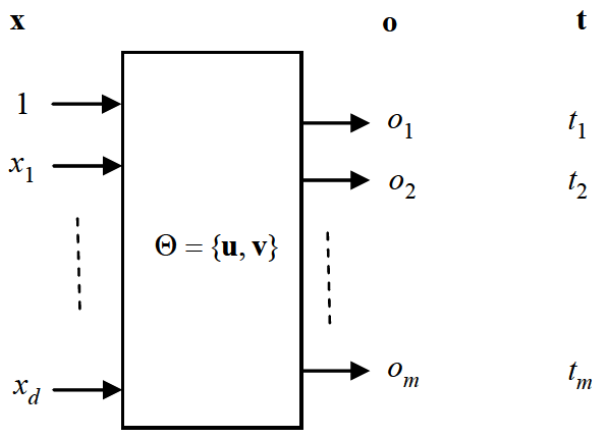
  - **input/output of k-th output node**

$$o_sum_k = \sum_{j=1}^{p} z_j\, v_{jk} + v_{0k} = \sum_{j=0}^{p} z_j\, v_{jk}$$

$$o_k = \tau_2(o_sum_k) = \tau_2\left(\sum_{j=0}^{p} z_j\, v_{jk}\right) = \tau_2\left(\sum_{j=0}^{p} \tau_1(z_sum_j)\, v_{jk}\right) = \tau_2\left(\sum_{j=0}^{p} \tau_1\left(\sum_{i=0}^{d} x_i\, u_{ij}\right) v_{jk}\right)$$

MLP - Learning

- **Step 2: Error computation**

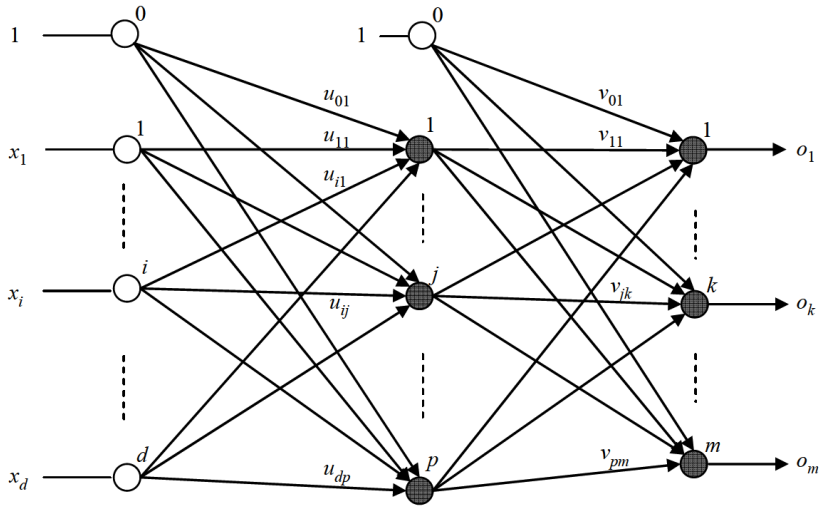


$$o_k = \tau_2\left(\sum_{j=0}^{p} z_j\, v_{jk}\right) = \tau_2\left(\sum_{j=0}^{p} \tau_1\left(\sum_{i=0}^{d} x_i\, u_{ij}\right) v_{jk}\right)$$

$$E = \frac{1}{2}\sum_{k=1}^{m}(o_k - t_k)^2 = \frac{1}{2}\sum_{k=1}^{m}\left(\tau_2\left(\sum_{j=0}^{p} z_j\, v_{jk}\right) - t_k\right)^2 = \frac{1}{2}\sum_{k=1}^{m}\left(\tau_2\left(\sum_{j=0}^{p} \tau_1\left(\sum_{i=0}^{d} x_i\, u_{ij}\right) v_{jk}\right) - t_k\right)^2$$

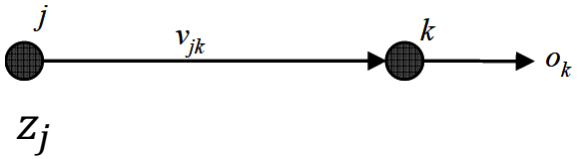
MLP - Learning

- **Step 3: weight updates by gradient descent method**
  - **Step 3-1:** $\mathrm{v}(h+1) = \mathrm{v}(h) - \rho \dfrac{\partial E}{\partial \mathrm{v}}, \quad v_{jk}(h+1) = v_{jk}(h) - \rho \dfrac{\partial E}{\partial v_{jk}}$
  - **Step 3-2:** $\mathrm{u}(h+1) = \mathrm{u}(h) - \rho \dfrac{\partial E}{\partial \mathrm{u}}, \quad u_{ij}(h+1) = u_{ij}(h) - \rho \dfrac{\partial E}{\partial u_{ij}}$



MLP - Learning

- $\frac{\partial E}{\partial v_{jk}}$ **for** v_{jk} **between hidden & output layer**



$$o_sum_k = f_1(v_{jk}) = \sum_{j=0}^{p} z_j \, v_{jk}$$

$$o_k = f_2(o_sum_k) = \tau_2(o_sum_k)$$

$$E = f_3(o_k) = \frac{1}{2}\sum_{k=1}^{m}(o_k - t_k)^2$$

$$\rightarrow \; E = f_3(f_2(f_1(v_{jk})))$$

$$\frac{\partial E}{\partial v_{jk}} = \frac{\partial f_3}{\partial v_{jk}} = \frac{\partial f_3}{\partial o_k} \times \frac{\partial f_2}{\partial o_sum_k} \times \frac{\partial f_1}{\partial v_{jk}}$$

$$= (o_k - t_k) \times \tau'(o_sum_k) \times z_j$$

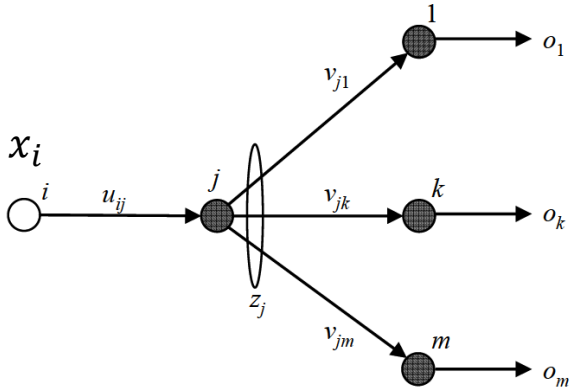
$$= \frac{\partial E}{\partial o_k} \times \tau'(o_sum_k) \times z_j$$

$$= \delta_k^{(2)} z_j$$

$$\delta_k^{(2)} = \frac{\partial E}{\partial o_k} \times \tau_2'(o_sum_k) = \frac{\partial E}{\partial o_sum_k}$$

MLP - Learning

- $\frac{\partial E}{\partial u_{ij}}$ **for** u_{ij} **between input & hidden layer**



$$z_sum_j = f_1(u_{ij}) = \sum_{i=0}^{d} x_i\, u_{ij}$$

$$z_j = f_2(z_sum_j) = \tau_1(z_sum_j)$$

$$o_sum_k = f_3(z_j) = \sum_{j=0}^{p} z_j\, v_{jk}$$

$$o_k = f_4(o_sum_k) = \tau_2(o_sum_k)$$

$$E = f_5(o_k) = \frac{1}{2}\sum_{k=1}^{m}(o_k - t_k)^2$$

$$\rightarrow\ E = f_5(f_4(f_3(f_2(f_1(u_{ij})))))$$

$$\frac{\partial E}{\partial u_{ij}} = \frac{\partial f_5}{\partial o_k} \times \frac{\partial f_4}{\partial o_sum_k} \times \frac{\partial f_3}{\partial z_j} \times \frac{\partial f_2}{\partial z_sum_j} \times \frac{\partial f_1}{\partial u_{ij}}$$

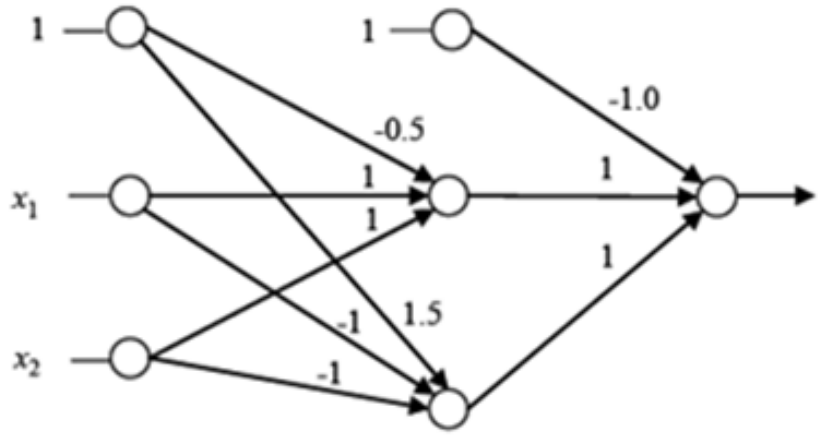
$$= \sum_{k=1}^{m}(o_k - t_k) \times \tau'(o_sum_k) \times v_{jk} \times \tau'(z_sum_j) \times x_i$$

$$= \sum_{k=1}^{m} \delta_k^{(2)} \times v_{jk} \times \tau'(z_sum_j) \times x_i$$

$$= \frac{\partial E}{\partial z_j} \times \tau'(z_sum_j) \times x_i = \delta_j^{(1)} x_i$$

Exercise

Consider the following Multi-Layer Perceptron.



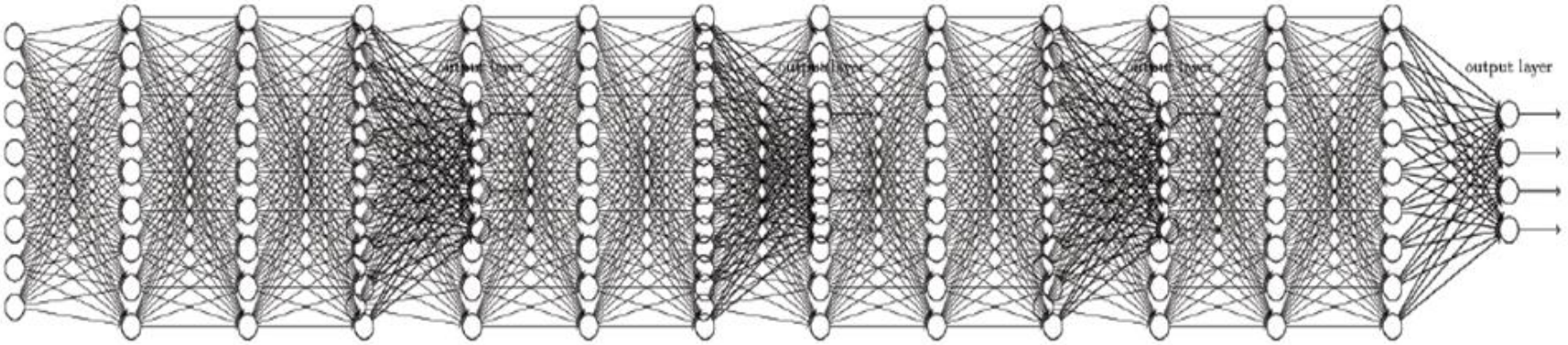
$$\tau(s) = \begin{cases} +1, s \geq 0 \\ -1, s < 0 \end{cases}$$

(1) Assuming that the activation function in each perceptron is a step function $\tau(s)$, what is the final output of this MLP for the pattern x=(0.5, 0.5)^T? Describe the processing sequence and specify the final output.

(2) Show the area of point P in $x_1 - x_2$ space where the MLP output for the P is -1.

MLP – Learning: Deep MLP

- **Deep MLP**



layer 1 *layer l* *layer L*

- x_i : i-th input value of layer 1
- $u_{ij}^{(l)}$: weight from i-th node of layer l to j-th node of layer $l+1$
- $z_j^{(l)}$: output from j-th node of layer l
- $o_k(t_k)$: k-th output(desired output) value in layer L

MLP – Learning: Deep MLP

- **Problem definition: given a set of N training samples,**

  $\mathbf{X}=\{ (\mathbf{x}_1, \mathbf{t}_1), (\mathbf{x}_2, \mathbf{t}_2), \ldots, (\mathbf{x}_N, \mathbf{t}_N) \}$,

  where $\mathbf{x}_i = [x_{i,1}, x_{i,2}, \ldots, x_{i,d}]^T$ and $\mathbf{t}_i = [t_{i,1}, t_{i,2}, \ldots, t_{i,m}]^T$

☞ **find *a possible* set of all weights $u_{ij}^{(l)}$ which map $\mathbf{x}_i$ to $\mathbf{t}_i$.**

- **Cost function for the goodness of $\theta = \left\{u_{ij}^{(l)}\right\}$**

$$E(\theta) = \frac{1}{2}\sum_{k=1}^{m}(o_k - t_k)^2$$

where m is the number of output nodes

BP for Deep MLP (1)

- **Step 1: Forward calculations with a training sample x**

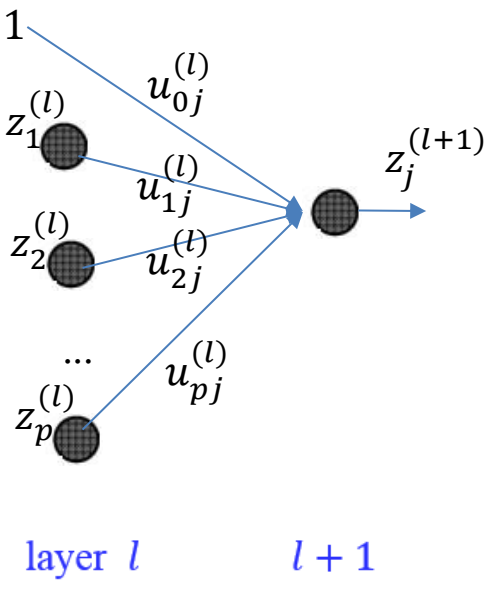
$$net_j^{(l+1)} = \sum_i z_i^{(l)} u_{ij}^{(l)}$$

$$z_j^{(l+1)} = \tau\left(net_j^{(l+1)}\right)$$

...

$$net_j^{(L)} = \sum_i z_i^{(L-1)} u_{ij}^{(L-1)}$$

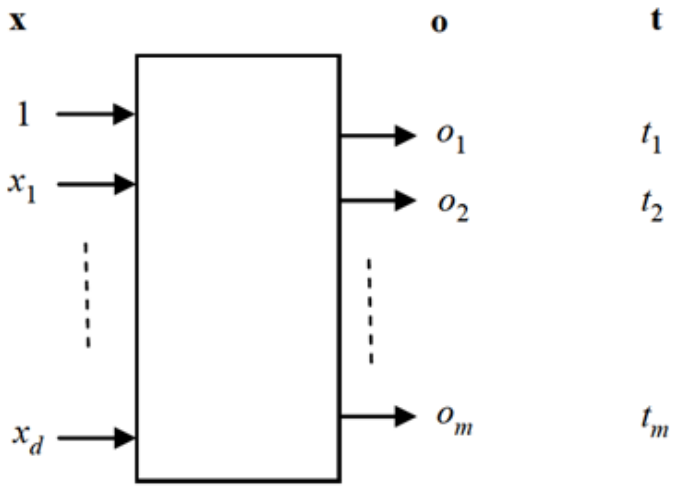
$$o_j = z_j^{(L)} = \tau\left(net_j^{(L)}\right)$$



BP for Deep MLP (2)

- **Step 2: Error computation**

$$E = \frac{1}{2}\sum_{k=1}^{m}(o_k - t_k)^2$$

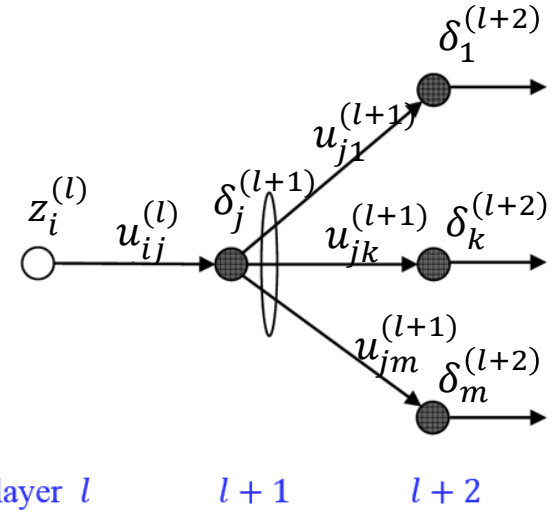


BP for Deep MLP (3)

- **Step 3: weight updates by error back-propagation**

$$u_{ij}^{(l)} = u_{ij}^{(l)} - \rho \frac{\partial E}{\partial u_{ij}^{(l)}}, \qquad l = L-1, L-2, \ldots, 1$$

$$\frac{\partial E}{\partial u_{ij}^{(l)}} = \frac{\partial E}{\partial net_j^{(l+1)}} \times \frac{\partial net_j^{(l+1)}}{\partial u_{ij}^{(l)}} = \delta_j^{(l+1)} \times z_i^{(l)}$$



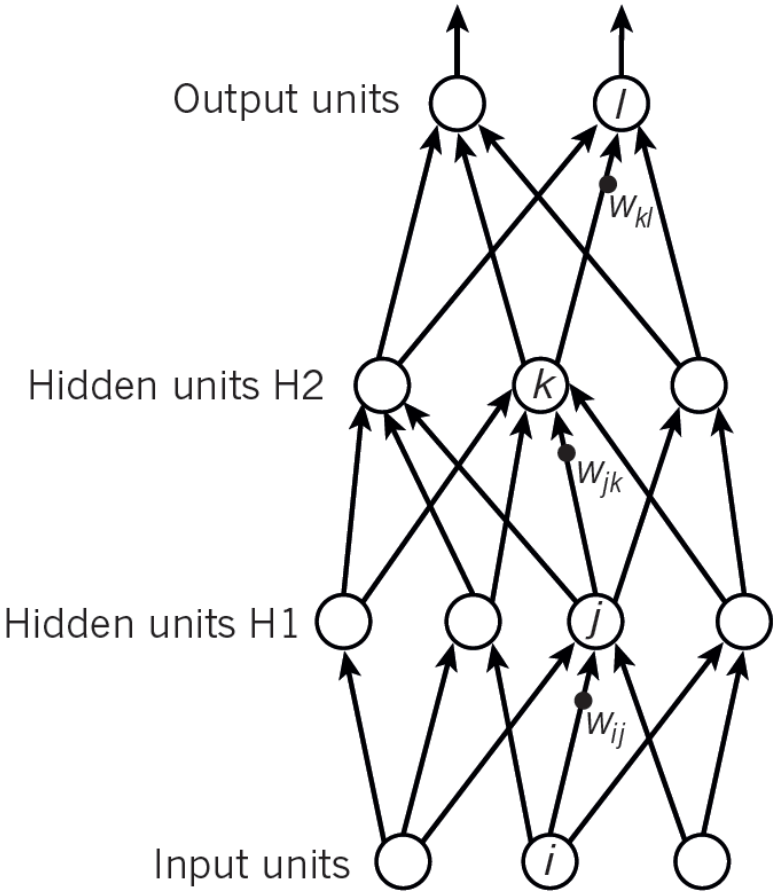
- **Error Back-propagation**

$$\delta_j^{(L)} = \tau'\left(net_j^{(L)}\right)\left(o_j - t_j\right) \qquad output\ layer$$

$$\delta_j^{(l+1)} = \tau'\left(net_j^{(l+1)}\right)\sum_k u_{jk}^{(l+1)}\delta_k^{(l+2)} \qquad non-output\ layer$$

MLP Deep Learning

- **Forward calculations [Nature, 2015]**



$$y_l = f(z_l)$$

$$z_l = \sum_{k \,\varepsilon\, \text{H2}} w_{kl}\, y_k$$

$$y_k = f(z_k)$$

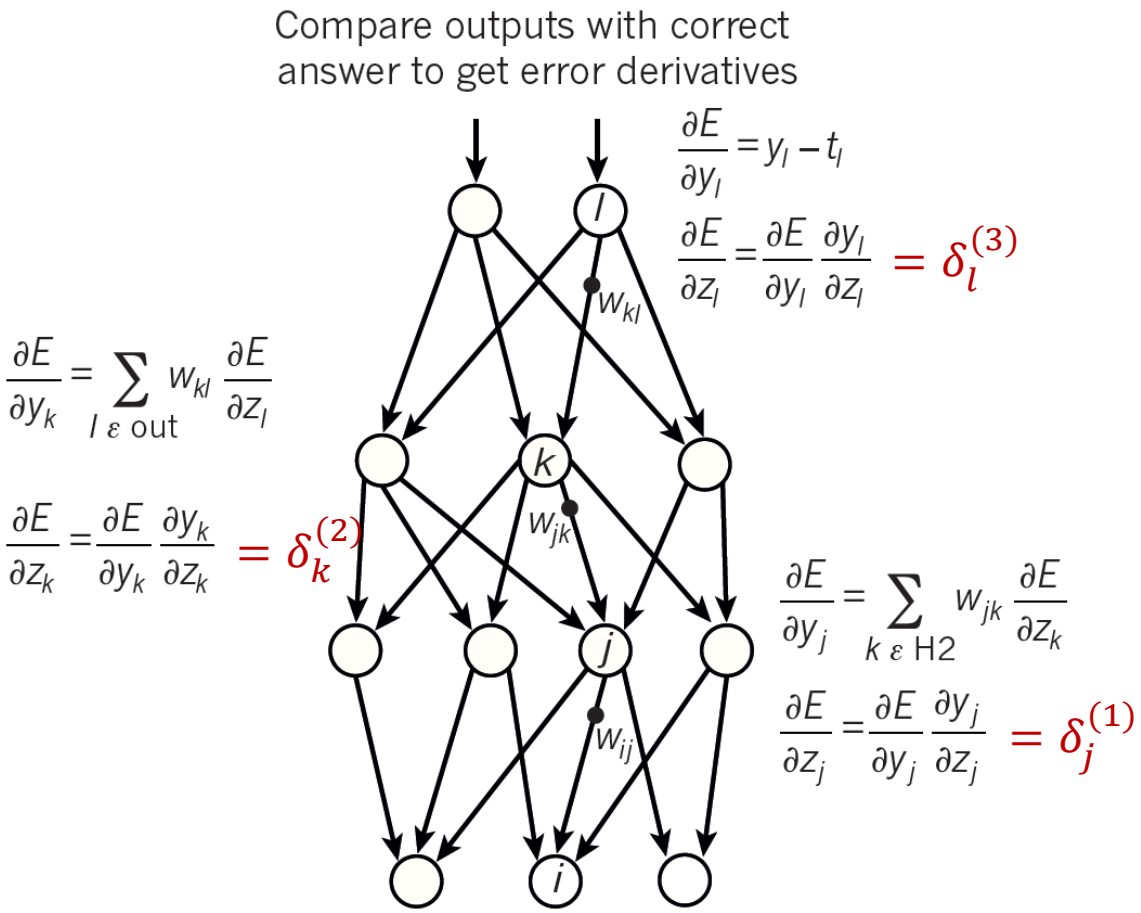
$$z_k = \sum_{j \,\varepsilon\, \text{H1}} w_{jk}\, y_j$$

$$y_j = f(z_j)$$

$$z_j = \sum_{i \,\varepsilon\, \text{Input}} w_{ij}\, x_i$$

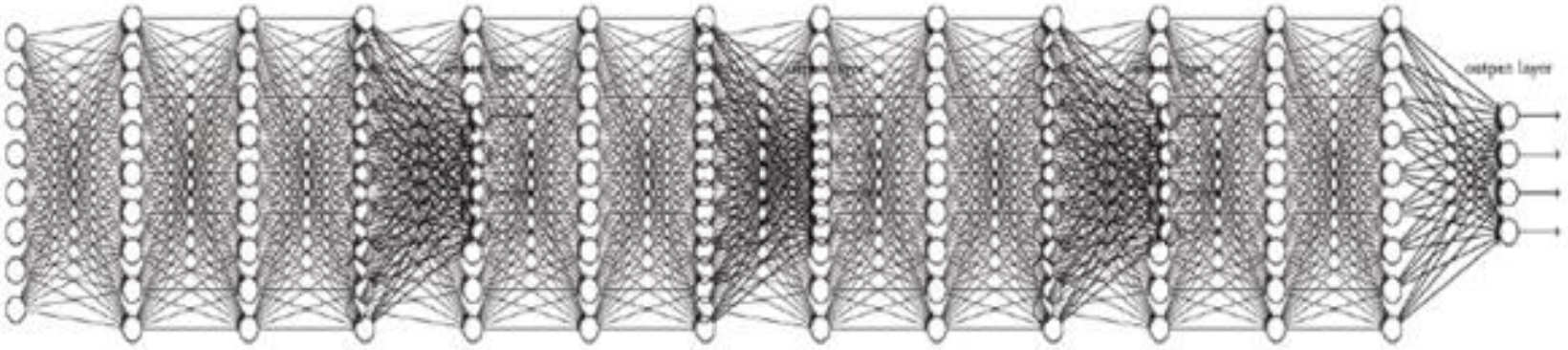
MLP Deep Learning

- **Backward error propagations [Nature, 2015]**



MLP – Breakthrough

- **Difficulties in Deep MLP (Geoffrey Hinton)**



- **We used the wrong type of nonlinearities (sigmoid → ReLU)**
- **We initialized the weights in a stupid way (random → pre-train)**
- **Our labeled datasets were thousands of times too small (BigData)**
- **Our computers were millions of times too slow (GPU)**